DSA Theory Notes for Placements

Core Data Structures & Algorithms Concepts Explained

1. Arrays & Strings

Arrays store elements in contiguous memory locations, allowing fast access using indexes. They are widely used because of their simplicity and cache efficiency. Common operations include traversal, insertion, deletion, searching, and sorting. String problems are often solved using techniques such as sliding window, two pointers, hashing, and prefix sums. Arrays are heavily asked in coding interviews because many advanced problems can be reduced to array manipulation.

2. Linked Lists

A linked list consists of nodes where each node stores data and a pointer to the next node. Unlike arrays, linked lists do not require contiguous memory. Types include singly linked lists, doubly linked lists, and circular linked lists. Common interview topics include reversing a linked list, detecting cycles using Floyd’s algorithm, merging lists, and finding middle nodes using slow and fast pointers.

3. Stack & Queue

A stack follows the Last In First Out (LIFO) principle, while a queue follows the First In First Out (FIFO) principle. Stacks are commonly used in recursion, expression evaluation, monotonic stack problems, and backtracking. Queues are used in BFS traversal, scheduling, buffering, and simulations. Variants include deque, priority queue, and circular queue.

4. Trees

Trees are hierarchical data structures consisting of nodes connected by edges. The topmost node is called the root. Binary trees have at most two children per node. Binary Search Trees maintain sorted order, enabling efficient searching. Important traversals include inorder, preorder, postorder, and level order traversal. Interview questions often involve recursion, depth-first search, breadth-first search, balancing, and Lowest Common Ancestor problems.

5. Graphs

Graphs are collections of vertices connected by edges. They can be directed or undirected, weighted or unweighted. Graph algorithms are extremely important for placements because they model networks, maps, and relationships. BFS is used for shortest paths in unweighted graphs, while DFS is useful for traversal and cycle detection. Dijkstra’s algorithm finds shortest paths in weighted graphs. Topological sorting is used for dependency resolution.

6. Recursion & Backtracking

Recursion occurs when a function calls itself to solve smaller subproblems. Every recursive solution requires a base case and recursive case. Backtracking is an advanced recursive technique used to explore all possibilities while discarding invalid paths. Common problems include permutations, combinations, Sudoku solving, and N-Queens. Understanding recursion trees and call stacks is essential.

7. Dynamic Programming

Dynamic Programming (DP) is used when problems have overlapping subproblems and optimal substructure. DP avoids repeated calculations by storing previously computed results. Memoization uses recursion with caching, while tabulation uses iterative bottom-up computation. Popular problems include Fibonacci, Knapsack, Longest Common Subsequence, and Longest Increasing Subsequence. DP is one of the most important topics in technical interviews.

8. Sorting & Searching

Sorting algorithms arrange data in a specific order. Common algorithms include Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort. Searching algorithms help locate elements efficiently. Linear search checks elements sequentially, while binary search works on sorted arrays with logarithmic complexity. Interviewers often test knowledge of time complexities and algorithm trade-offs.

9. Time & Space Complexity

Time complexity measures how the running time of an algorithm grows with input size. Space complexity measures additional memory usage. Big O notation is used to represent complexity classes such as O(1), O(log n), O(n), O(n log n), and $O(n^2)$. Understanding complexity analysis helps compare solutions and optimize performance during interviews.